AI-Powered
CI/CD Security Pipeline

Complete Project Plan, Architecture & Implementation Guide

GitHub Actions	AI Security	AWS ECS
CI/CD	Claude/Gemini/Beadrock/OpenAI	Cloud Deploy

GitLeaks | Semgrep SAST | Trivy CVE Scan | Docker | AWS ECR | ECS Fargate

AI-Powered CI/CD Security Pipeline

This document outlines a complete project plan, architecture, and implementation guide for building an AI-augmented DevSecOps pipeline using GitHub Actions, Claude/Gemini/Beadrock/OpenAI API, and AWS ECS Fargate.

Core Objective: To create a production-grade CI/CD pipeline that automatically scans every Pull Request (PR) for security issues using open-source tools (GitLeaks, Semgrep SAST, Trivy CVE Scanner), synthesizes the findings with the Claude AI, and posts a plain-English security verdict. Crucially, the pipeline is designed to **block merges** for any PR flagged with a **CRITICAL** risk or **BLOCK MERGE** recommendation by the AI.

Pipeline Architecture (Three Jobs):

1. **Job 1: Security Scan:** Runs on every PR. Executes tests, GitLeaks (secrets), Semgrep (SAST), and Trivy (CVEs). It then feeds the results to the Claude AI Reviewer script, which posts the verdict as a PR comment and fails the job if the risk is Critical/Block Merge.
2. **Job 2: Build & Push:** Runs only on successful merges/pushes to the `main` branch. Builds the Docker image, tags it with the commit SHA, and pushes it to AWS ECR.
3. **Job 3: Deploy:** Runs only after a successful push to ECR. Triggers a zero-downtime rolling deployment to AWS ECS Fargate by forcing a new deployment.

Implementation Phases (6 Phases/2 Weeks): The guide breaks down the implementation into six phases, starting with environment setup (Python FastAPI app, Dockerfile, ECR repo) and security tool configuration. The central Phase 4 is dedicated to building the `ai_security_review.py` script for Claude integration. Phase 5 puts everything together in the `pipeline.yml` GitHub Actions file, and Phase 6 covers end-to-end testing and verification.

Section 1 — Project Overview & Objectives	**Page 1**
1. What Are We Building?	
1.1 Core Value Proposition	
1.2 Three-Job Pipeline Architecture	
Section 2 — Project Plan & Timeline	**Page 2**
2. Project Plan	
2.1 Prerequisites Checklist	
2.2 Complete Tech Stack	
Section 3 — Complete Pipeline Workflow	**Page 3**
3.1 Trigger Conditions	
3.2 Job 1: Security Scan — All 9 Steps	
3.3 Job 2: Build & Push (main branch only)	
3.4 Job 3: Deploy to ECS Fargate	
3.5 The Claude AI Review — How It Works Exactly	
Section 4 — Full Implementation Guide	**Page 5**
4. Phase 1 — Environment Setup & App Scaffolding	
5. Phase 2 — Containerization	
6. Phase 3 — Security Tool Configuration	
7. Phase 4 — Claude AI Reviewer	
8. Phase 5 — GitHub Actions Pipeline YAML	
Section 5 — Testing, Verification & Portfolio Polish	**Page 9**
9. Phase 6 — Testing the Pipeline End-to-End	

Section 1 — Project Overview & Objectives

1. What Are We Building?

A production-grade, AI-augmented DevSecOps pipeline that automatically scans every pull request for security vulnerabilities, uses the Claude AI API to generate a plain-English security verdict, and blocks risky code from ever reaching production.

Unlike a standard CI/CD project, this one has an AI brain. It reads scan output from three security tools, synthesizes the findings, and posts a human-readable risk assessment directly on the GitHub PR. If Claude says BLOCK MERGE, the pipeline fails and the code cannot be merged.

1.1 Core Value Proposition

- Catches hardcoded secrets before they hit Git history (GitLeaks)
- Finds code-level vulnerabilities and injection flaws automatically (Semgrep SAST)
- Detects known CVEs in Docker images before they reach production (Trivy)
- Synthesizes all findings into a plain-English risk verdict (Claude API)
- Blocks high-risk PRs from merging — zero manual security review needed
- Auto-deploys clean builds to AWS ECS Fargate on every merge to main

1.2 Three-Job Pipeline Architecture

The pipeline runs as three sequential GitHub Actions jobs with strict conditions. Security scanning runs on every PR. Build and push only runs on merges to main. Deploy runs after a successful push. Nothing untested ever reaches the cloud.

Job 1: Security Scan	Job 2: Build & Push	Job 3: Deploy
Every PR trigger	Merge to main only	After push success

Section 2 — Project Plan & Timeline

2. Project Plan

The project is organized into 6 phases over 2 weeks. Each phase has a clear goal and concrete deliverables. Complete phases 1-3 before touching the pipeline YAML.

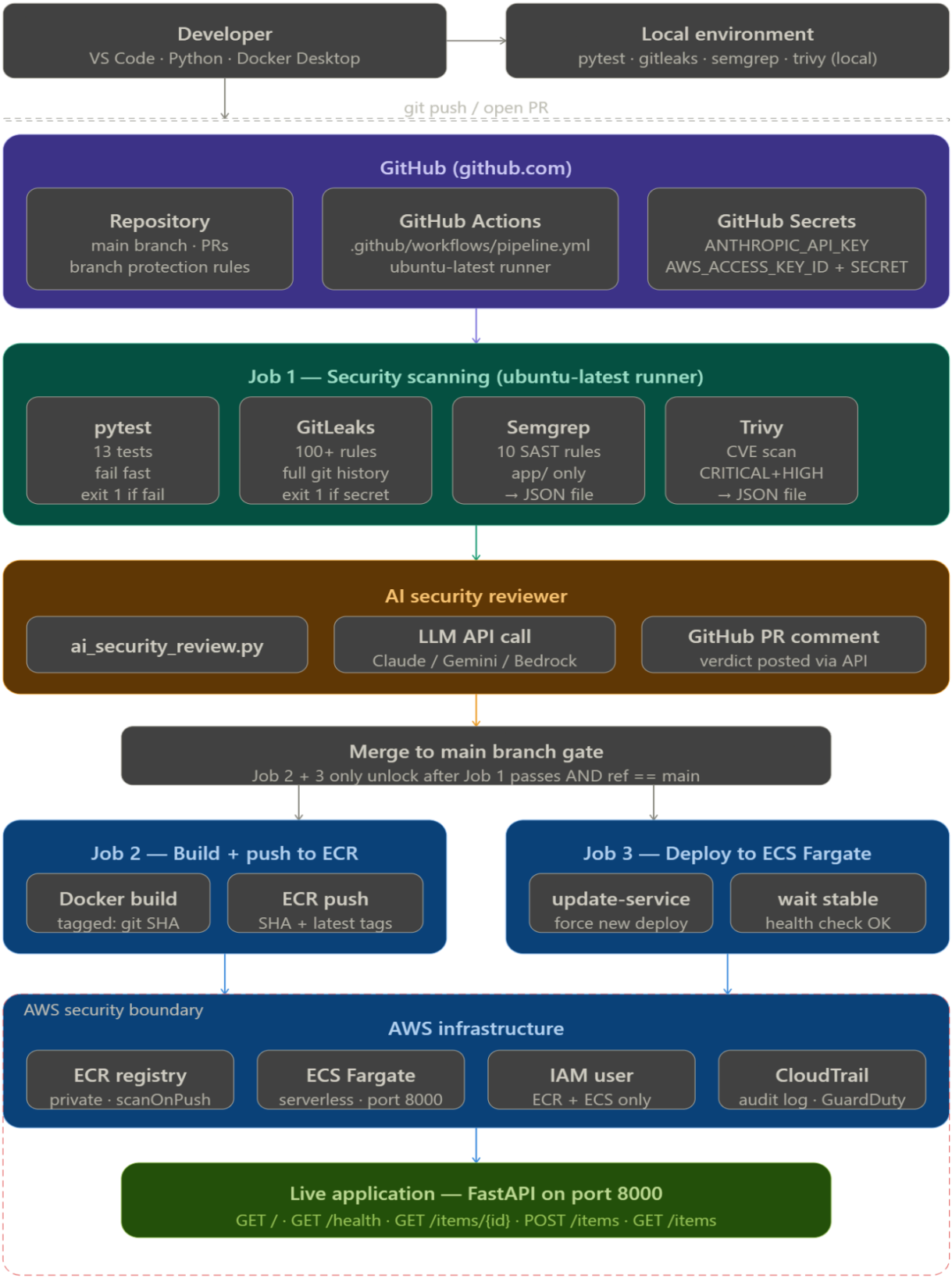
Phase	Name	Duration	Deliverable
Phase 1	Environment Setup & App Scaffolding	Day 1-2	Working FastAPI app with passing tests
Phase 2	Containerization	Day 2-3	Production Dockerfile + ECR repo created
Phase 3	Security Tool Configuration	Day 3-5	GitLeaks + Semgrep + Trivy all configured
Phase 4	Claude AI Reviewer Script	Day 5-7	AI review posting verdict as PR comment
Phase 5	GitHub Actions Pipeline YAML	Day 7-10	Full pipeline running end-to-end
Phase 6	Testing & Portfolio Polish	Day 10-14	Live demo + README + screenshots ready

2.1 Prerequisites Checklist

- GitHub account (free)
- AWS account — free tier is sufficient for ECR and ECS Fargate
- Anthropic API key — get at console.anthropic.com (free credits available)
- Docker Desktop installed on your machine
- Python 3.11+ installed
- AWS CLI configured with a dedicated IAM user (ECR push + ECS update permissions only)

2.2 Complete Tech Stack

Tool	Category	Purpose
GitHub Actions	CI/CD	Orchestrates the entire pipeline on PR open and push events
Docker	Container	Packages the app; Trivy scans the resulting image for CVEs
AWS ECR	Cloud	Private container registry — stores images tagged by commit SHA
AWS ECS Fargate	Cloud	Serverless container runtime — deploys the app with zero downtime
GitLeaks	Security	Scans full git history for hardcoded secrets and API keys
Semgrep	Security	Static analysis — catches SQL injection, insecure patterns, bad practices
Trivy	Security	Scans Docker images for known CVEs at CRITICAL and HIGH severity
Claude API	AI	Synthesizes all scan results into a plain-English PR security verdict
Python FastAPI	Code Quality	The sample app the pipeline builds, scans, and deploys
pytest	Code Quality	Unit tests that must pass before any security scanning begins


Developer
VS Code · Python · Docker Desktop
Local environment
pytest · gitleaks · semgrep · trivy (local)
git push / open PR
GitHub (github.com)
Repository
main branch · PRs
branch protection rules
GitHub Actions
.github/workflows/pipeline.yml
ubuntu-latest runner
GitHub Secrets
ANTHROPIC_API_KEY
AWS_ACCESS_KEY_ID + SECRET
Job 1 — Security scanning (ubuntu-latest runner)
pytest
13 tests
fail fast
exit 1 if fail
GitLeaks
100+ rules
full git history
exit 1 if secret
Semgrep
10 SAST rules
app/ only
→ JSON file
Trivy
CVE scan
CRITICAL+HIGH
→ JSON file
AI security reviewer
ai_security_review.py
LLM API call
Claude / Gemini / Bedrock
GitHub PR comment
verdict posted via API
Merge to main branch gate
Job 2 + 3 only unlock after Job 1 passes AND ref == main
Job 2 — Build + push to ECR
Docker build
tagged: git SHA
ECR push
SHA + latest tags
Job 3 — Deploy to ECS Fargate
update-service
force new deploy
wait stable
health check OK
AWS security boundary
AWS infrastructure
ECR registry
private · scanOnPush
ECS Fargate
serverless · port 8000
IAM user
ECR + ECS only
CloudTrail
audit log · GuardDuty
Live application — FastAPI on port 8000
GET / · GET /health · GET /items/{id} · POST /items · GET /items

Section 3 — Complete Pipeline Workflow

3. Full Pipeline Workflow

This section walks through every step the pipeline executes, in exact order. Understanding this flow lets you explain the architecture.

3.1 Trigger Conditions

The pipeline has two trigger events that activate different sets of jobs:

Trigger	Jobs That Execute
Pull Request opened or updated	Job 1: security-scan only (includes AI review posted to PR)
Push or merge to main branch	Job 1 then Job 2 then Job 3 — full pipeline including deploy

3.2 Job 1: Security Scan — All 9 Steps

1. Checkout with fetch-depth 0 — full git history so GitLeaks can scan all commits, not just the latest.
2. Set up Python 3.11 and install all app dependencies from requirements.txt.
3. Run pytest — any failing test halts the pipeline immediately. No partial scanning.
4. GitLeaks scans the entire git history using your custom .gitleaks.toml ruleset. Any secret found = immediate failure.
5. Semgrep runs .semgrep.yml rules against the app/ directory and outputs results to semgrep-results.json.
6. Docker builds the image tagged scan-target:pr. This is a local build for scanning only — not pushed to ECR yet.
7. Trivy scans the Docker image for CRITICAL and HIGH severity CVEs. exit-code is 0 here — the AI step decides whether to fail.
8. AI Review (PR only): Python script reads all three JSON result files, sends them plus the git diff to Claude API. Claude returns a structured verdict. The script posts it as a PR comment via the GitHub REST API.
9. If Claude's response contains CRITICAL or BLOCK MERGE, the script calls sys.exit(1), marking the job as failed and blocking the merge.

3.3 Job 2: Build & Push (main branch only)

Condition

This job only runs when pushing to main AND Job 1 passed. PRs never trigger an ECR push.

10. Configure AWS credentials from GitHub Secrets using the official aws-actions action.
11. Login to Amazon ECR.
12. Build the Docker image tagged with the Git commit SHA — every image is uniquely traceable.
13. Push the tagged image to ECR. It is now available for deployment.

3.4 Job 3: Deploy to ECS Fargate

Condition

Runs only after Job 2 succeeds. Triggers a rolling ECS deployment — new tasks start, pass health checks, old tasks drain. Zero downtime.

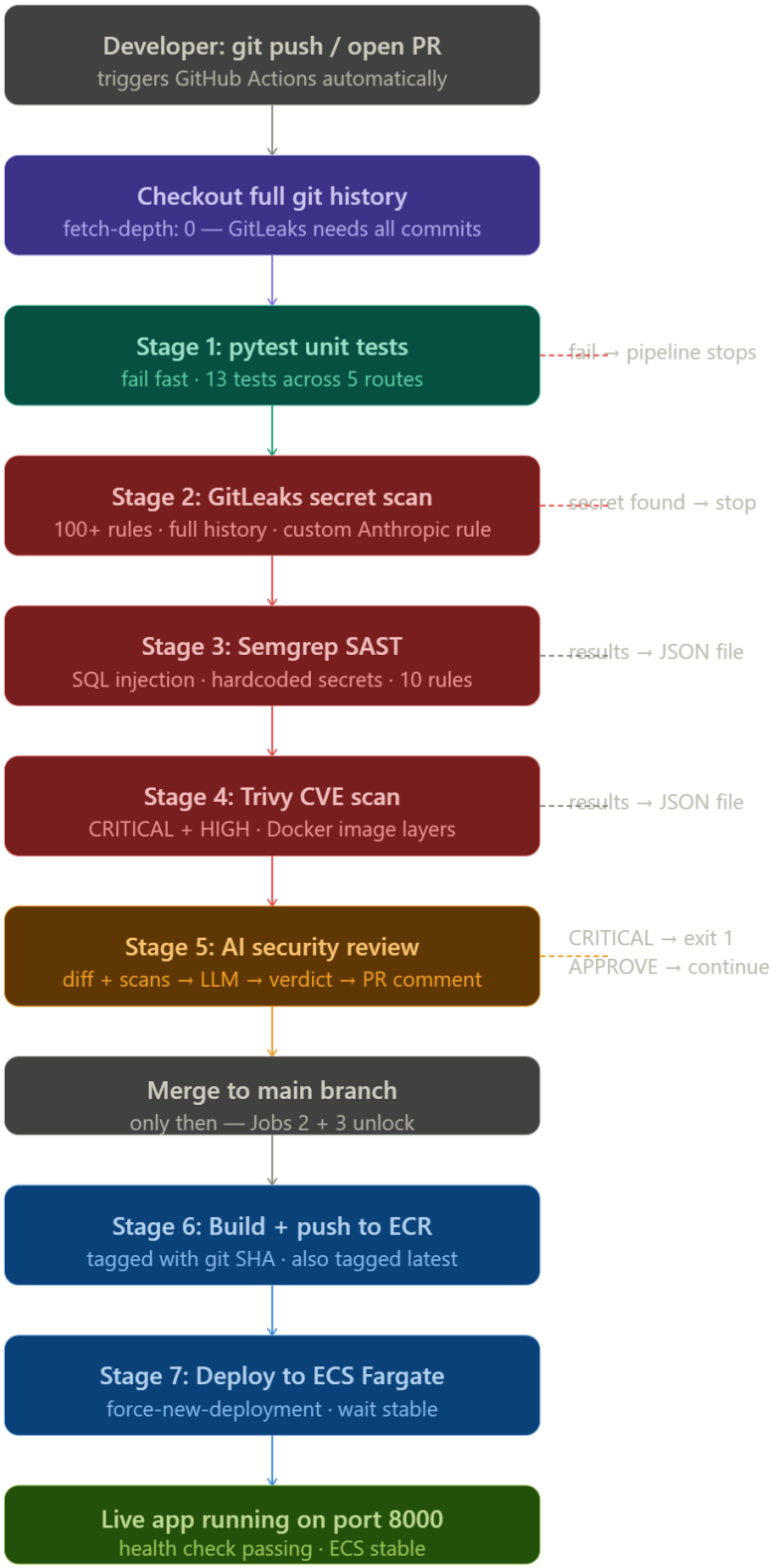
14. Configure AWS credentials again (each job runs in a completely fresh environment).
15. Run aws ecs update-service with --force-new-deployment flag.
16. ECS pulls the latest image from ECR, starts new tasks, health-checks them, and drains the old tasks.

3.5 The AI Review — How It Works Exactly

The Anthropic API integration is the most novel part. Here is the exact logic of the ai_security_review.py script:

- Runs git diff origin/main...HEAD --stat to get a compact summary of what changed in this PR (capped at 3000 chars).
- Reads trivy-results.json, semgrep-results.json, and gitleaks-results.json — whichever exist after the scan steps.
- Constructs a prompt that includes the diff summary and all scan JSON, asking Claude for: RISK LEVEL, SUMMARY, TOP ISSUES, RECOMMENDATION.
- Calls anthropic.messages.create() with model claude-sonnet-4-5 and max_tokens=800.
- Posts the response as a GitHub PR comment using the GitHub REST API (authenticated with GITHUB_TOKEN).
- Checks if the response text contains CRITICAL or BLOCK MERGE — if yes, calls sys.exit(1) to fail the pipeline job.

Diagram:


Developer: git push / open PR
triggers GitHub Actions automatically
Checkout full git history
fetch-depth: 0 — GitLeaks needs all commits
Stage 1: pytest unit tests
fail fast · 13 tests across 5 routes
fail → pipeline stops
Stage 2: GitLeaks secret scan
100+ rules · full history · custom Anthropic rule
secret found → stop
Stage 3: Semgrep SAST
SQL injection · hardcoded secrets · 10 rules
results → JSON file
Stage 4: Trivy CVE scan
CRITICAL + HIGH · Docker image layers
results → JSON file
Stage 5: AI security review
diff + scans → LLM → verdict → PR comment
CRITICAL → exit 1
APPROVE → continue
Merge to main branch
only then — Jobs 2 + 3 unlock
Stage 6: Build + push to ECR
tagged with git SHA · also tagged latest
Stage 7: Deploy to ECS Fargate
force-new-deployment · wait stable
Live app running on port 8000
health check passing · ECS stable

Section 4 — Full Implementation Guide

4. Phase 1 — Environment Setup & App Scaffolding

Step 1: Install all required tools

```
# macOS
brew install gh awscli aquasecurity/trivy/trivy gitleaks
pip install semgrep anthropic fastapi uvicorn pytest httpx

# Verify
trivy --version && gitleaks version && semgrep --version
```

Step 2: Create the project structure

```
mkdir ai-cicd-pipeline && cd ai-cicd-pipeline
git init
mkdir -p app tests .github/workflows scripts
```

Step 3: app/main.py — the FastAPI application

```
from fastapi import FastAPI
app = FastAPI()

@app.get('/')
def read_root():
    return {'message': 'Hello from secure AI pipeline!'}

@app.get('/health')
def health():
    return {'status': 'ok'}
```

Step 4: tests/test_main.py

```
from fastapi.testclient import TestClient
from app.main import app
client = TestClient(app)
```

```
def test_root():
    response = client.get('/')
    assert response.status_code == 200

def test_health():
    assert client.get('/health').json() == {'status': 'ok'}
```

Step 5: requirements.txt

```
fastapi==0.111.0
uvicorn==0.30.0
httpx==0.27.0
pytest==8.2.0
anthropic>=0.28.0
```

5. Phase 2 — Containerization

Step 6: Dockerfile — production-grade

```
FROM python:3.11-slim

# Non-root user for security
RUN useradd -m -u 1000 appuser
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ ./app/

USER appuser
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 7: .dockerignore

```
.git
.github
__pycache__
*.pyc
.env*
*.key
*.pem
tests/
```

Step 8: Create AWS ECR repository

```
aws ecr create-repository \
  --repository-name ai-cicd-app \
  --region us-east-1 \
  --image-scanning-configuration scanOnPush=true

# Save your AWS account ID for later
aws sts get-caller-identity --query Account --output text
```

Step 9: Build and test locally

```
docker build -t ai-cicd-app:local .
docker run -p 8000:8000 ai-cicd-app:local
curl http://localhost:8000/health
```

```
# Expected: {"status": "ok"}

# Preview what Trivy will find
trivy image ai-cicd-app:local
```

6. Phase 3 — Security Tool Configuration

Step 10: .gitleaks.toml — secret scanning rules

```
[extend]
useDefault = true

[[rules]]
id = "custom-anthropic-key"
description = "Anthropic API Key"
regex = '''sk-ant-[a-zA-Z0-9\-_]{40,}'''
tags = ["key", "anthropic"]

[allowlist]
regexes = ['''EXAMPLE_KEY''', '''sk-ant-REPLACE_ME''']
```

Step 11: .semgrep.yml — SAST rules

```
rules:
  - id: hardcoded-password
    pattern: password = "..."
    message: Hardcoded password detected
    languages: [python]
    severity: ERROR

  - id: sql-injection-risk
    pattern: cursor.execute($QUERY + $INPUT)
    message: Use parameterized queries instead
    languages: [python]
    severity: ERROR

  - id: debug-true
    pattern: DEBUG = True
    message: Debug mode enabled in production code
    languages: [python]
    severity: WARNING
```

Step 12: Test security tools locally

```
# Test GitLeaks
gitleaks detect --source . --config .gitleaks.toml --verbose

# Test Semgrep
semgrep --config .semgrep.yml app/

# Test Trivy
trivy image ai-cicd-app:local --severity CRITICAL,HIGH
```

7. Phase 4 — AI Reviewer

Step 13: scripts/ai_security_review.py — full script

```
#!/usr/bin/env python3
import os, sys, json, subprocess
import anthropic

def get_pr_diff():
    result = subprocess.run(
        ['git', 'diff', 'origin/main...HEAD', '--stat'],
        capture_output=True, text=True)
    return result.stdout[:3000]

def read_scan_results():
    results = {}
    for fname in ['trivy-results.json',
                  'semgrep-results.json',
                  'gitleaks-results.json']:
        if os.path.exists(fname):
            with open(fname) as f:
                results[fname] = f.read()[:2000]
    return results

def review_with_claude(diff, scan_results):
    client = anthropic.Anthropic()
    scan_summary = '\n\n'.join(
        f'=== {k} ===\n{v}' for k, v in scan_results.items())
    prompt = f'''You are a senior DevSecOps engineer.
PR Changes (git diff stat):
{diff}

Security Scan Results:
{scan_summary}

Respond with:
1. RISK LEVEL: (CRITICAL / HIGH / MEDIUM / LOW / CLEAN)
2. SUMMARY: 2-3 sentences plain English
3. TOP ISSUES: Up to 3 most important findings
4. RECOMMENDATION: BLOCK MERGE or APPROVE WITH NOTES or APPROVE
Use BLOCK MERGE if CRITICAL issues are found.'''
    msg = client.messages.create(
        model='claude-sonnet-4-5',
        max_tokens=800,
        messages=[{'role': 'user', 'content': prompt}])
    return msg.content[0].text

def post_pr_comment(review_text):
    import urllib.request
    token = os.environ['GITHUB_TOKEN']
    repo  = os.environ['GITHUB_REPOSITORY']
    pr    = os.environ['PR_NUMBER']
    url   = f'https://api.github.com/repos/{repo}/issues/{pr}/comments'
    body  = f'## AI Security Review\n\n{review_text}\n\n*Reviewed by Claude*'
```

```
    data  = json.dumps({'body': body}).encode()
    req = urllib.request.Request(
        url, data=data,
        headers={'Authorization': f'Bearer {token}',
                 'Content-Type': 'application/json'})
    urllib.request.urlopen(req)

if __name__ == '__main__':
    diff   = get_pr_diff()
    scans  = read_scan_results()
    review = review_with_claude(diff, scans)
    print(review)
    if os.environ.get('PR_NUMBER'):
        post_pr_comment(review)
    if 'CRITICAL' in review or 'BLOCK MERGE' in review:
        print('AI review flagged critical issues. Failing build.')
        sys.exit(1)
```

8. Phase 5 — GitHub Actions Pipeline YAML

Step 14: .github/workflows/pipeline.yml — complete file

```yaml
name: AI-powered secure CI/CD pipeline

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: ai-cicd-app
  ECS_SERVICE: ai-cicd-service
  ECS_CLUSTER: ai-cicd-cluster

jobs:
  security-scan:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt
      - run: pytest tests/ -v
      - uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          GITLEAKS_CONFIG: .gitleaks.toml
      - run: |
          pip install semgrep
          semgrep --config .semgrep.yml app/ \
            --json > semgrep-results.json || true
      - run: docker build -t scan-target:pr .
      - uses: aquasecurity/trivy-action@master
        with:
          image-ref: scan-target:pr
          format: json
          output: trivy-results.json
          severity: CRITICAL,HIGH
          exit-code: '0'
      - name: AI security review (Claude)
        if: github.event_name == 'pull_request'
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

```
          PR_NUMBER: ${{ github.event.number }}
          GITHUB_REPOSITORY: ${{ github.repository }}
        run: python scripts/ai_security_review.py

  build-and-push:
    needs: security-scan
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}
      - id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2
      - env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
          IMAGE_TAG: ${{ github.sha }}
        run: |
          docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
          docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG

  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}
      - run: |
          aws ecs update-service \
            --cluster ${{ env.ECS_CLUSTER }} \
            --service ${{ env.ECS_SERVICE }} \
            --force-new-deployment
```

Step 15: Add GitHub Secrets

Go to your repo on GitHub → Settings → Secrets and variables → Actions → New repository secret.

Secret Name	Where to Get It
`ANTHROPIC_API_KEY`	sk-ant-... from console.anthropic.com
`AWS_ACCESS_KEY_ID`	From your IAM user — minimal permissions only
`AWS_SECRET_ACCESS_KEY`	Paired with the access key ID above

Section 5 — Testing, Verification & Portfolio Polish

9. Phase 6 — Testing the Pipeline End-to-End

Step 16: First push and PR

```
git add .
git commit -m 'feat: AI-powered CI/CD pipeline initial setup'

# Create repo at github.com/new first, then:
git remote add origin https://github.com/YOUR_USERNAME/ai-cicd-pipeline.git
git push -u origin main

# Open a test PR
git checkout -b feature/test-pipeline
echo '# First test' >> README.md
git add . && git commit -m 'test: trigger AI pipeline review'
git push origin feature/test-pipeline
# Open a PR on GitHub UI — watch the Actions tab
```

Step 17: Test that security detection works

Safe testing procedure

Add a FAKE secret string (not a real key) to verify detection. Confirm Claude flags it in the PR comment. Then revert. Never use a real API key for this test.

```
# Add fake secret to trigger detection
echo 'DEBUG_KEY = "sk-ant-FAKE_FOR_TESTING_1234567890abcdef"' >> app/main.py
git add . && git commit -m 'test: verify secret detection works'
git push origin feature/test-pipeline

# GitLeaks should fail the job
# Claude comment should say BLOCK MERGE

# Revert after verifying
git revert HEAD && git push origin feature/test-pipeline
```

Step 18: Expected Claude PR comment

```
## AI Security Review
```

```
**RISK LEVEL:** LOW

**SUMMARY:** The PR makes minor documentation changes. No
vulnerabilities were detected by Trivy, Semgrep, or GitLeaks.
The Docker image uses python:3.11-slim with no critical CVEs.

**TOP ISSUES:** None identified.

**RECOMMENDATION:** APPROVE

*Reviewed by Claude via Anthropic API*
```

Verification checklist — confirm all of these

- GitHub Actions tab: all 3 jobs show as green checkmarks
- PR has an automated comment from Claude with a security verdict
- AWS ECR console shows a new image tagged with the commit SHA
- AWS ECS console shows a new deployment in progress or complete
- curl http://your-alb-url/health returns {"status": "ok"}
- Fake secret test: GitLeaks fails the job AND Claude says BLOCK MERGE